MOUNT KENYA UNIVERSITY

DEPARTMENT OF ENTERPRISE COMPUTING

PROJECT DOCUMENTATION

UNIT CODE: BIT3208

UNIT NAME: ADVANCED WEB DESIGN

Presented By

NAME: CLAIRE KIHWAGA

REG NO: BBIT/2025/33474

LECTURER: MR NYORO

SEMESTER: MAY-AUGUST

GITHUB REPOSITORY LINK: https://github.com/Claire-kaki/BIT3208_Project

PROJECT DETAILS

Project title: ELECHUB

Selected Technologies:

- PHP
- MYSQL

Contents

WEEK1: Local Environment Setup 1

FIGURE1: INSTALLATION OF XAMPP 1
FIGURE2: APACHE AND MYSQL RUNNING 2
FIGURE3: LOCAL HOST TEST PAGE 2
FIGURE4: HELLO WORLD TEST 2
FIGURE5: DATABASE CONNECTION TEST 3

WEEK2: WIREFRAMES AND GUI DESIGN 4

FIGURE1: HAND-DRAWN WIREFRAME 4
FIGURE2: DASHBOARD LAYOUT DESIGN 4
FIGURE3: MOBILE VIEW MOCKUP 6
FIGURE4: COLOR THEME AND NAVIGATION STRUCTURE 7
FIGURE5: GUI PROTOTYPE IN DRAW.IO 8

WEEK3: JAVASCRIPT AND PHP BASICS 9

FIGURE1: JAVASCRIPT FORM VALIDATION 9
FIGURE2: PASSWORD STRENGTH CHECKER 9
FIGURE3: PHP SYNTAX PRACTICE 9
FIGURE4: DATABASE CONNECTION SCRIPT 10
FIGURE5: DYNAMIC USER INPUT HANDLING 10

WEEK4: PHP FORM HANDLING AND VALIDATION 11

FIGURE1: LOGIN FORM & FORM VALIDATION 11
FIGURE2: PHP PROCESSING 11
FIGURE4: DATABASE CONNECTION 12
FIGURE5: GITHUB UPDATES 13

WEEK5: DATABASE CREATION AND CRUD OPERATIONS 14

FIGURE1: DATABASE& TABLE CREATION 14
FIGURE2: CRUD OPERATIONS 15
FIGURE3: SQL QUERY EXECUTION 15
FIGURE4: LOGIN DATABASE INTEGRATION 15
FIGURE5: PHP DATABASE CONNECTION 16

WEEK1: Local Environment Setup
Installation and Testing of Local Development Environment

FIGURE1: INSTALLATION OF XAMPP

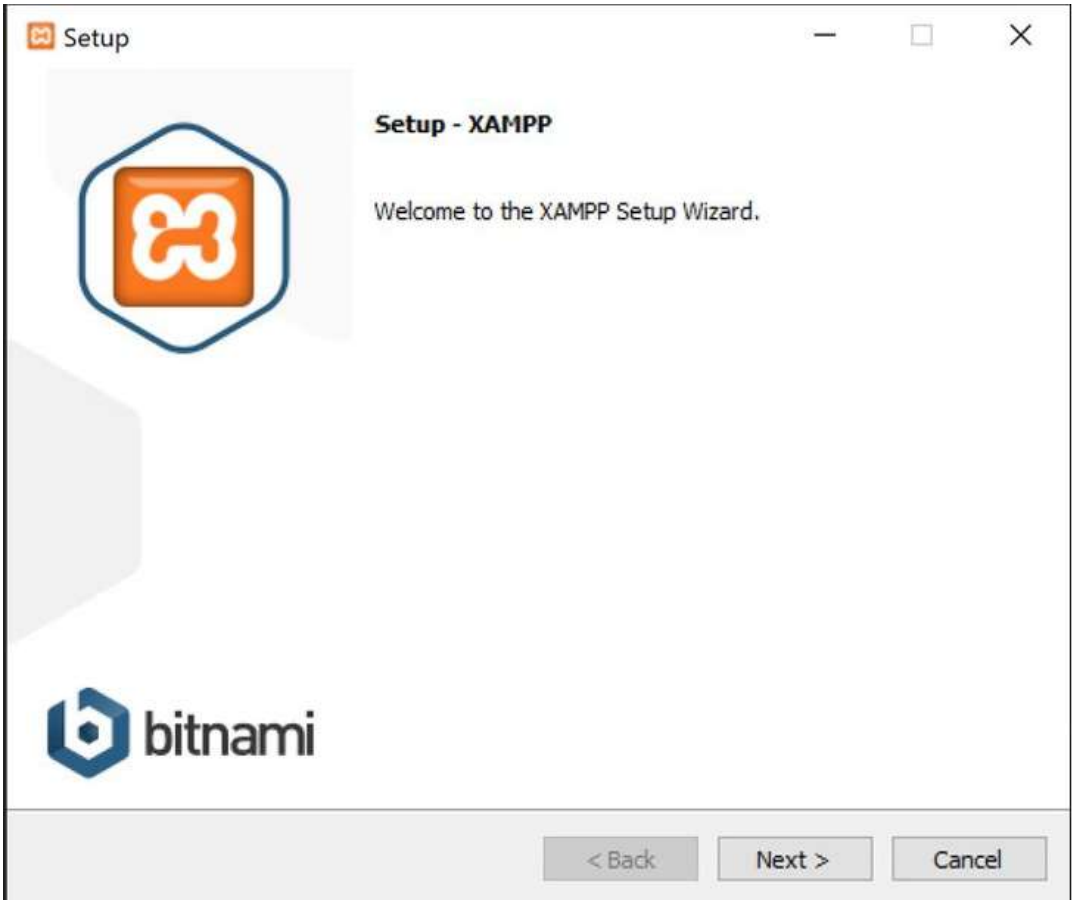


FIGURE2: APACHE AND MYSQL RUNNING

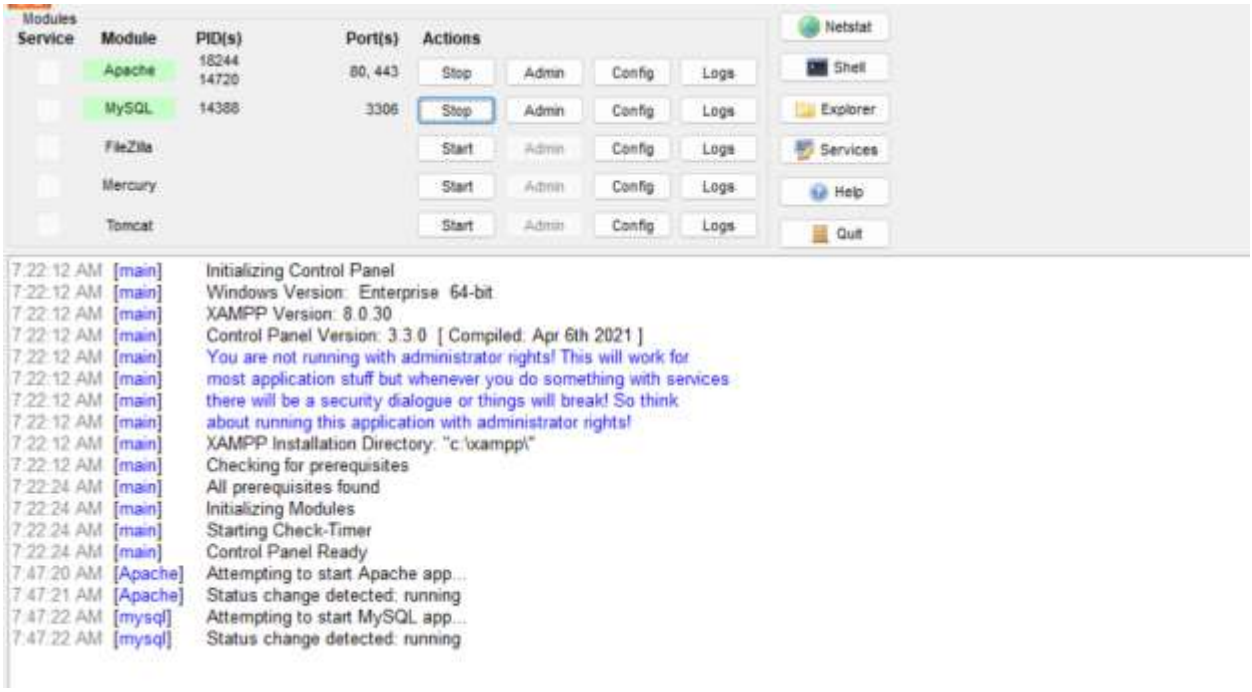


FIGURE3: LOCAL HOST TEST PAGE

FIGURE4: HELLO WORLD TEST

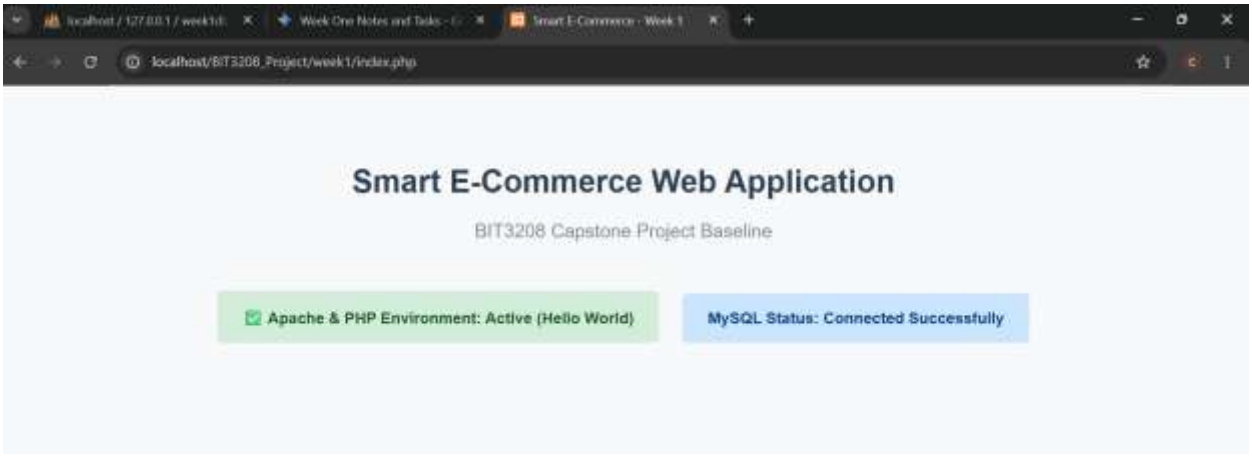
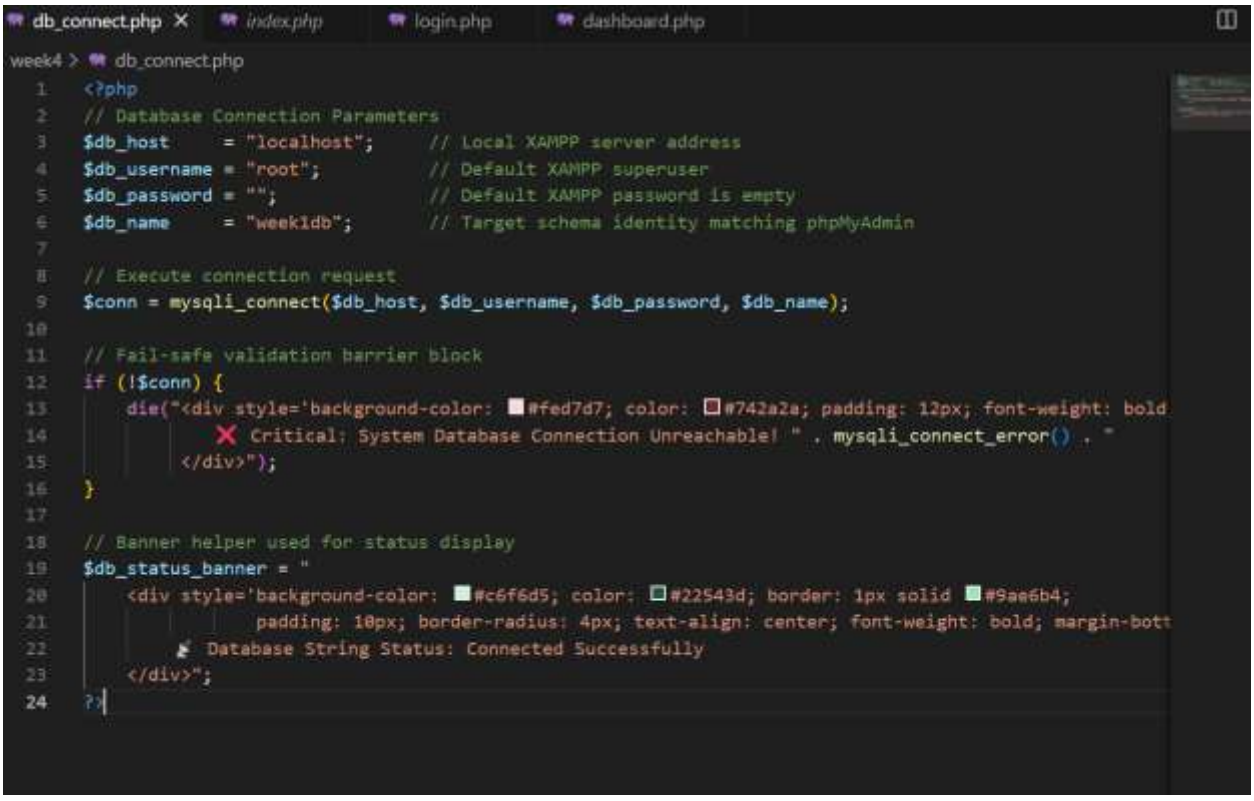


FIGURE5: DATABASE CONNECTION TEST



DESCRIPTION

During this week I installed and setup XAMPP as my local server engine. Technologies used are: xampp local server, HTML5 and CSS3. I used CSS to layout boundaries, padding and margins.

Figure 5 shows successful database connection

WEEK2: WIREFRAMES AND GUI DESIGN
USER INTERFACE PLANNING AND SYSTEM DESIGN

FIGURE1: HAND-DRAWN WIREFRAME

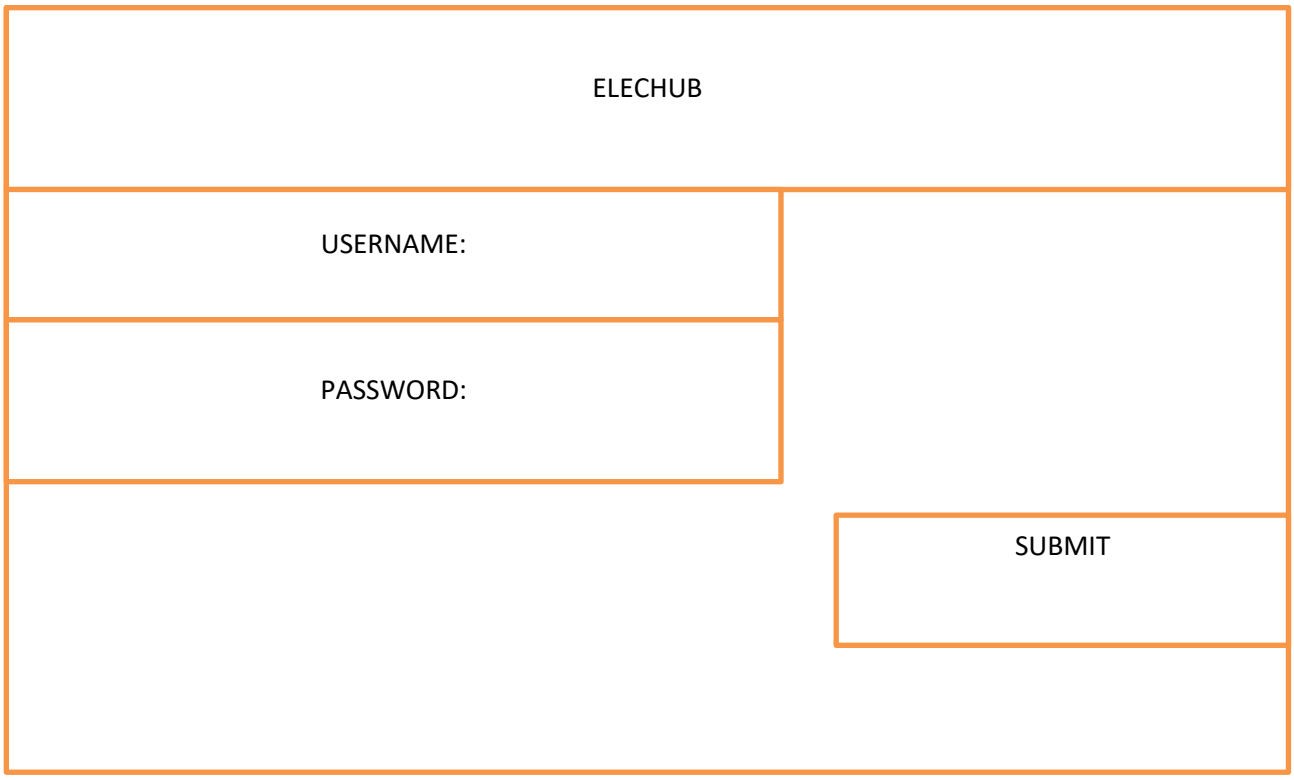


FIGURE2: DASHBOARD LAYOUT DESIGN

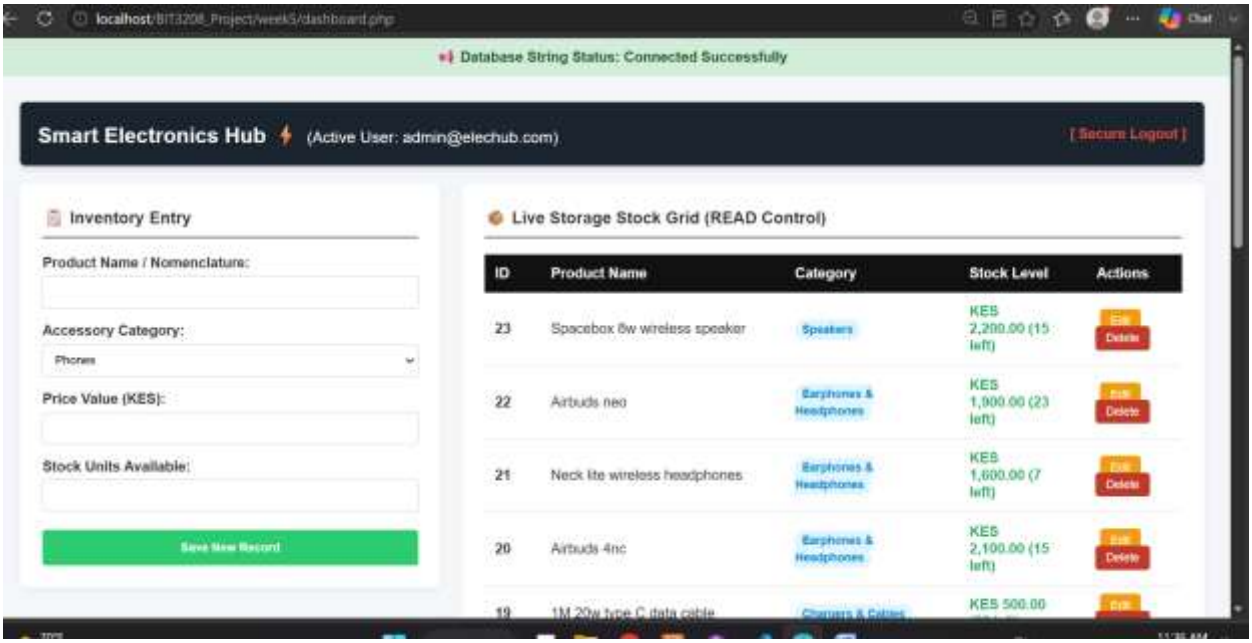

localhost/BIT3208_Project/week5/dashboard.php
Database String Status: Connected Successfully
Smart Electronics Hub
(Active User: admin@elechub.com)
[Secure Logout]
Inventory Entry
Product Name / Nomenclature:
Accessory Category:
Phones
Price Value (KES):
Stock Units Available:
Save New Record
Live Storage Stock Grid (READ Control)
ID
Product Name
Category
Stock Level
Actions
23
Spacebox 6w wireless speaker
Speakers
KES 2,200.00 (15 left)
Edit
Delete
22
Airbuds neo
Earphones & Headphones
KES 1,900.00 (23 left)
Edit
Delete
21
Neck lite wireless headphones
Earphones & Headphones
KES 1,600.00 (7 left)
Edit
Delete
20
Airbuds 4nc
Earphones & Headphones
KES 2,100.00 (15 left)
Edit
Delete
19
1M 20w type C data cable
Chargers & Cables
KES 500.00
Edit

FIGURE3: MOBILE VIEW MOCKUP

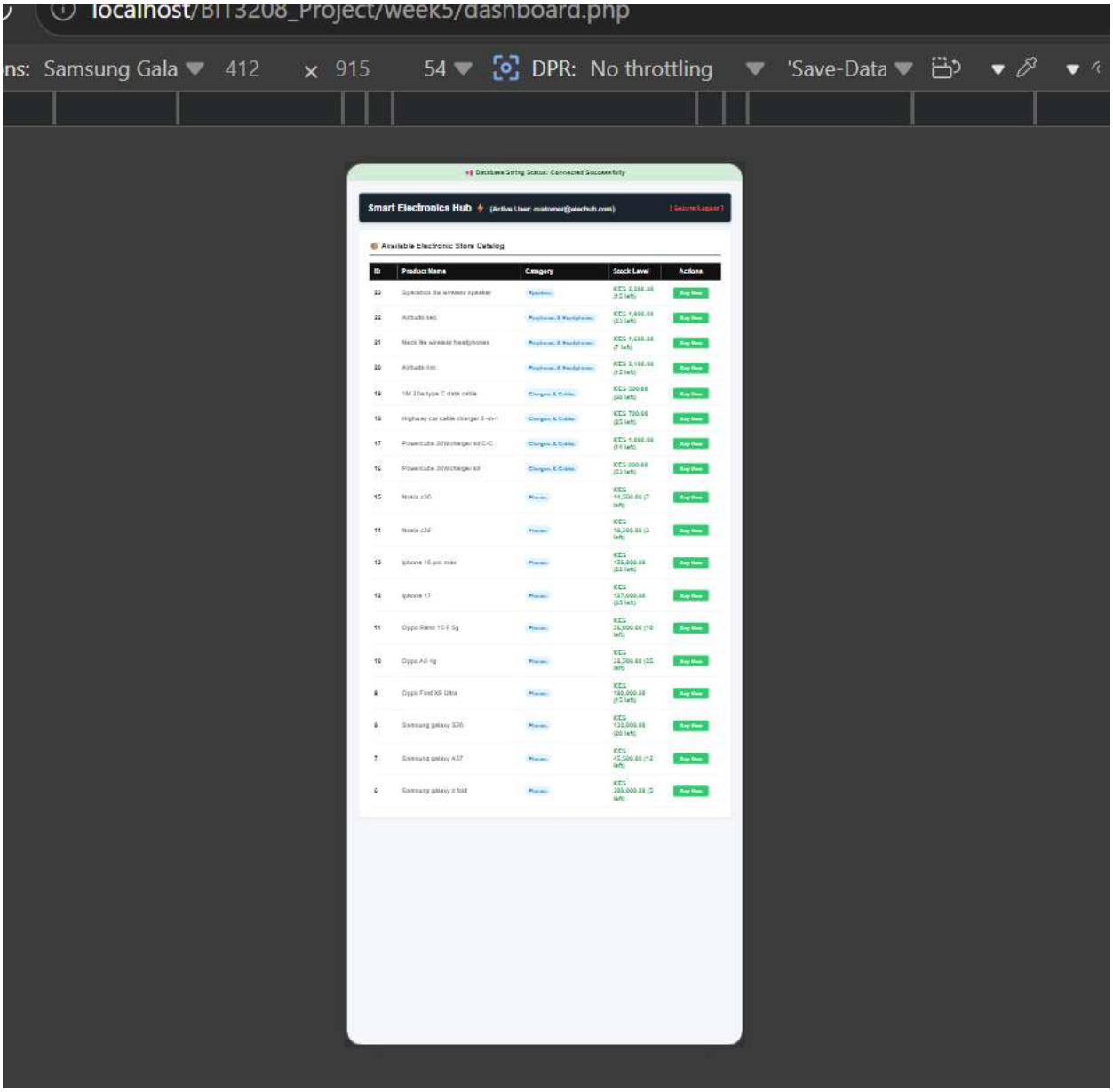


FIGURE4: COLOR THEME AND NAVIGATION STRUCTURE

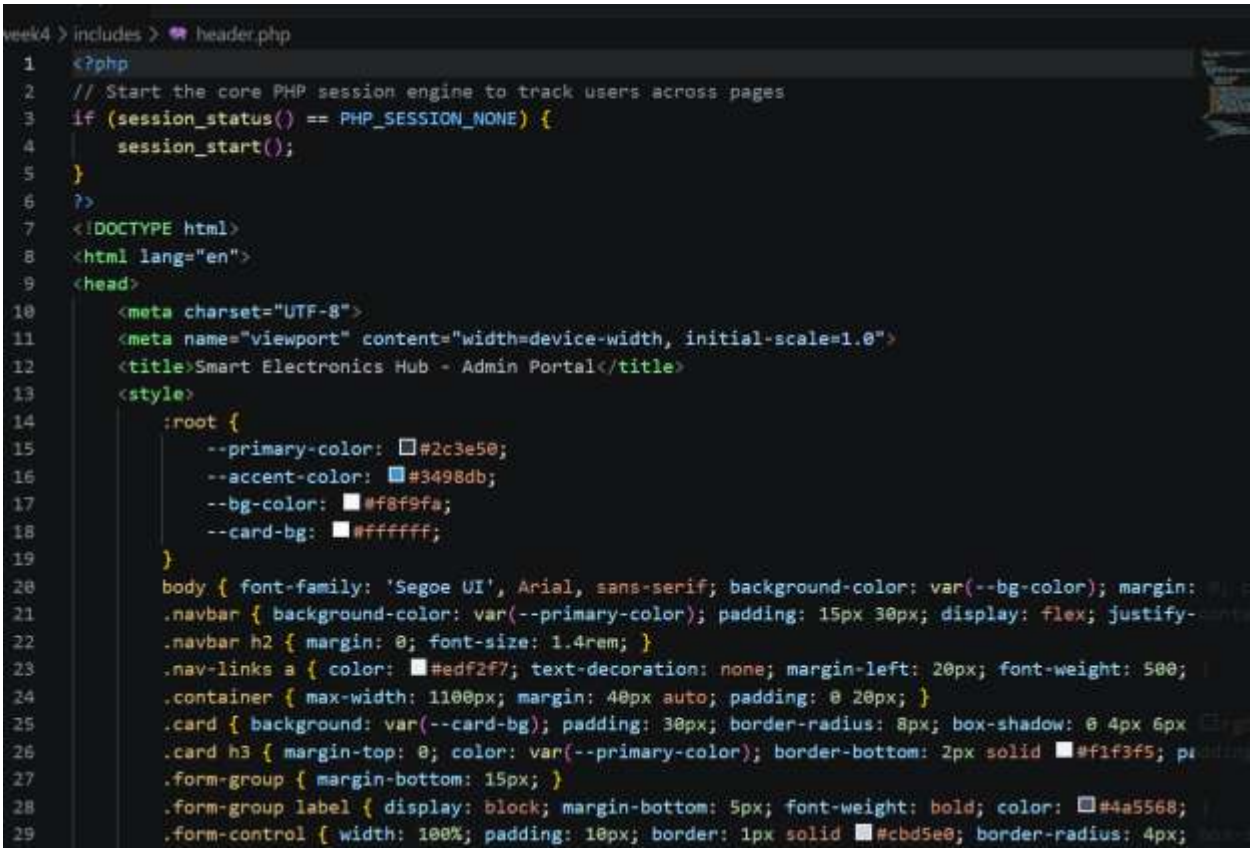
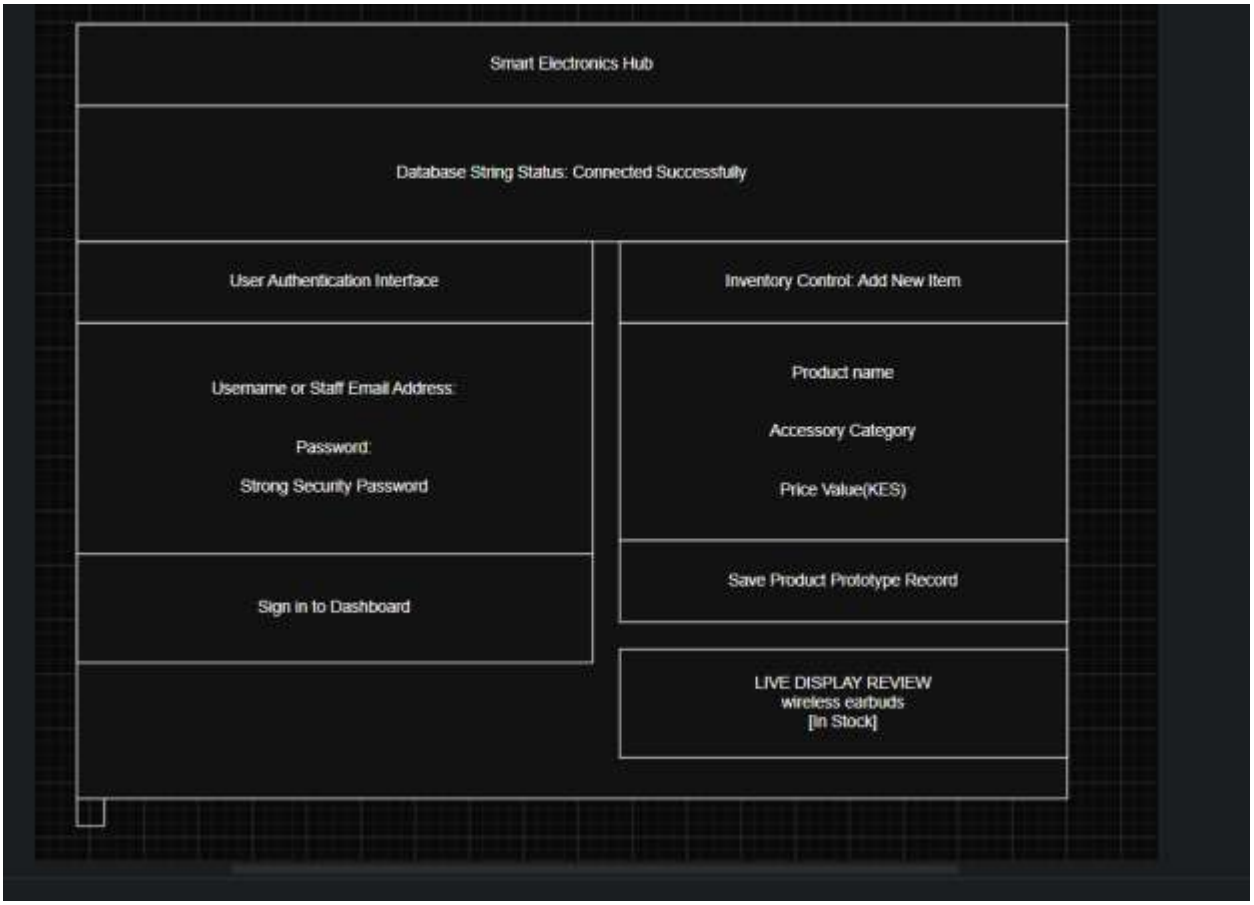


FIGURE5: GUI PROTOTYPE IN DRAW.IO



DESCRIPTION

This week I built a wireframe using draw.io to map out a layout for my interface. Figure 2 shows dashboard layout

Figure 3 shows how the interface reflows neatly onto mobile screens.

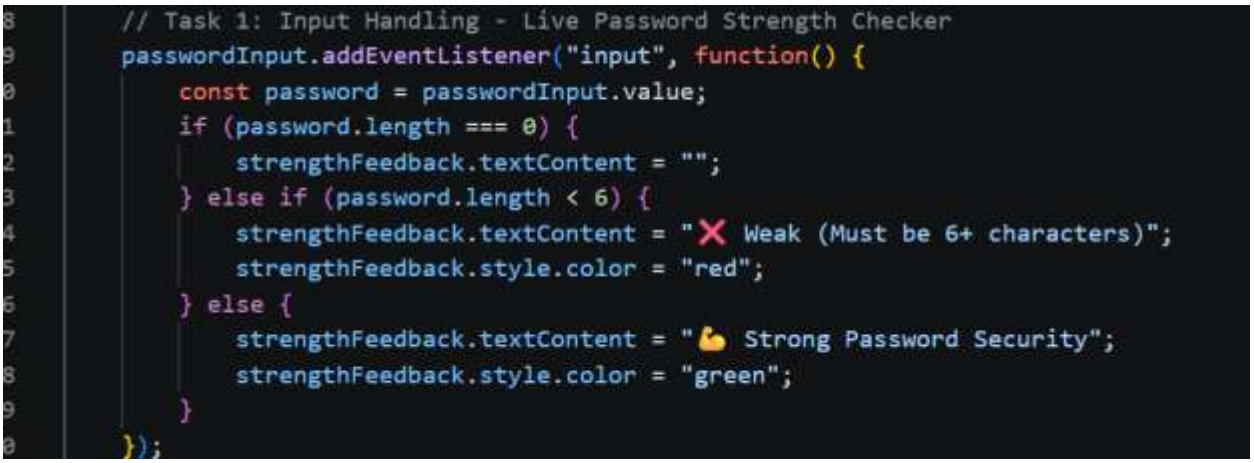
Figure 4 shows the planned palette selections and background tone to prevent eye strain.

WEEK3: JAVASCRIPT AND PHP BASICS
FRONTEND INTERACTION AND BACKEND FOUNDATIONS

FIGURE1: JAVASCRIPT FORM VALIDATION

```
// Task 1: Form Validation System on Submit
loginForm.addEventListener("submit", function(event) {
    const email = document.getElementById("adminEmail").value;
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

FIGURE2: PASSWORD STRENGTH CHECKER



```
// Task 1: Input Handling - Live Password Strength Checker
passwordInput.addEventListener("input", function() {
    const password = passwordInput.value;
    if (password.length === 0) {
        strengthFeedback.textContent = "";
    } else if (password.length < 6) {
        strengthFeedback.textContent = "❌ Weak (Must be 6+ characters)";
        strengthFeedback.style.color = "red";
    } else {
        strengthFeedback.textContent = "💪 Strong Password Security";
        strengthFeedback.style.color = "green";
    }
});
```

FIGURE3: PHP SYNTAX PRACTICE

week3 > index.php

```
<?php
// Task 4: Basic PHP Database Connection String Practice
$host = "localhost";
$user = "root";
$pass = "";
$db   = "week1db"; // Uses your established local database schema

$conn = mysqli_connect($host, $user, $pass, $db);
$db_status = "Connected Successfully";
if (!$conn) {
    $db_status = "Connection Failed: " . mysqli_connect_error();
}
?>
```

FIGURE4: DATABASE CONNECTION SCRIPT

```
7
8   $conn = mysqli_connect($host, $user, $pass, $db);
9   $db_status = "Connected Successfully";
10  if (!$conn) {
11      $db_status = "Connection Failed: " . mysqli_connect_error();
```

FIGURE5: DYNAMIC USER INPUT HANDLING

```
112             <h3>User Authentication Interface</h3>
113             <form id="loginForm" action="" method="POST">
114                 <div class="form-group">
115                     <label>Username or Staff Email Address</label>
116                     <input type="text" id="adminEmail" class="form-control" placeholder="e.g., ac
117                 </div>
118                 <div class="form-group">
119                     <label>Password</label>
120                     <input type="password" id="adminPassword" class="form-control" placeholder="E
121                     <small id="passwordStrength" style="display:block; margin-top:5px; font-weigh
122                 </div>
123                 <button type="submit" class="btn">Sign In to Dashboard</button>
124             </form>
125         </div>
126
127         <div class="card">
```

DESCRIPTION

This week was all about creating a frontend and backend screen so as to ensure client-server interactivity. For the client side I used JavaScript to handle user events. For the server side I used PHP to handle script logic and string variables.

On figure 1 is a screenshot of JavaScript form validation that helps detect mistakes on wrong email format.

Figure 2 shows password strength checker which using a length password condition turns red when the password is less than 6 to show weak password , and turns green when password has more than 6 characters to suggest strong password.

WEEK4: PHP FORM HANDLING AND VALIDATION
DYNAMIC BACKEND PROCESSING AND REAL USER INTERACTION SYSTEMS

FIGURE1: LOGIN FORM & FORM VALIDATION

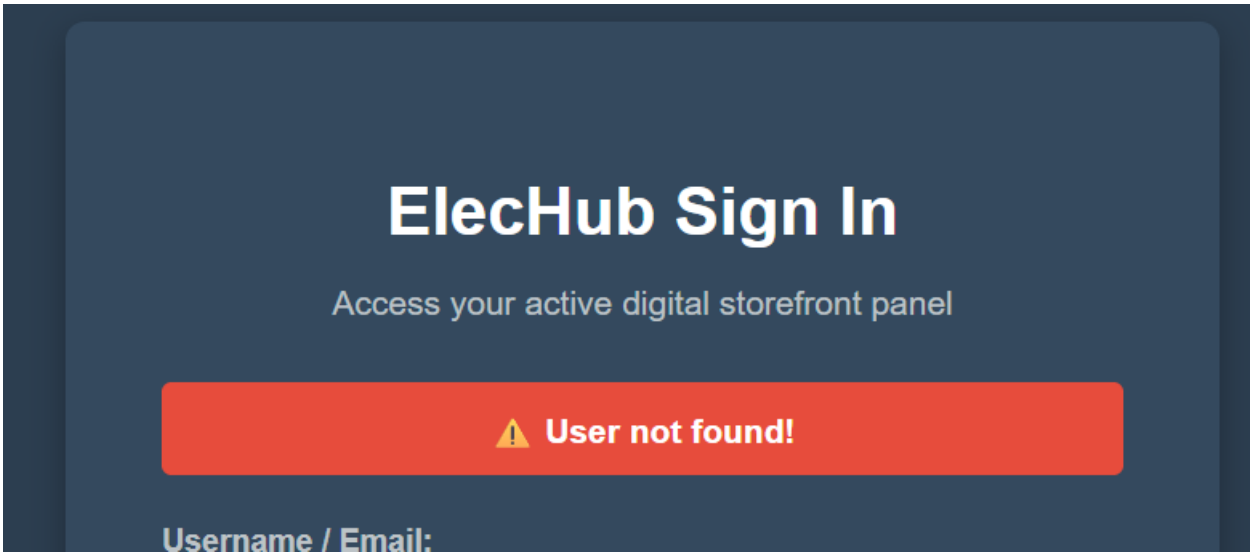


FIGURE2: PHP PROCESSING

```php
<?php
session_start();
// Security Check: Redirect to login if user session isn't active
if (!isset($_SESSION['user'])) {
    header("Location: login.php");
    exit();
}
```

FIGURE3: FOLDER STRUCTURE

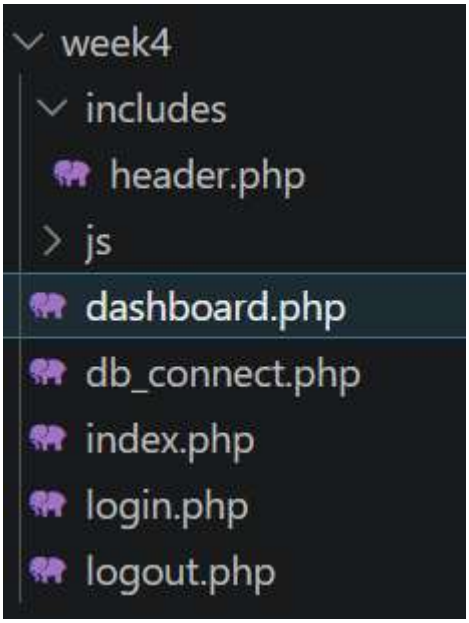


FIGURE4: DATABASE CONNECTION

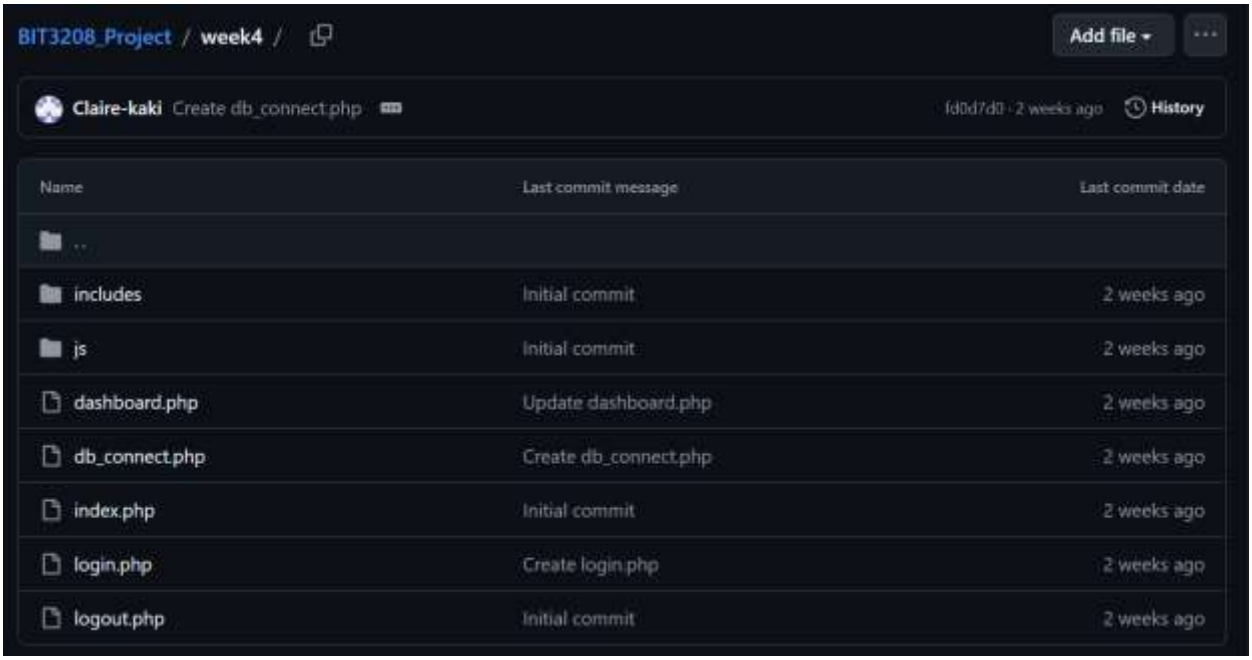
```php
<?php
// Database Connection Parameters
$db_host     = "localhost";     // Local XAMPP server address
$db_username = "root";          // Default XAMPP superuser
$db_password = "";              // Default XAMPP password is empty
$db_name     = "week1db";       // Target schema identity matching phpMyAdmin

// Execute connection request
$conn = mysqli_connect($db_host, $db_username, $db_password, $db_name);

// Fail-safe validation barrier block
if (!$conn) {
    die("<div style='background-color: #fed7d7; color: #742a2a; padding: 12px; font-weight: bold
            ❌ Critical: System Database Connection Unreachable! " . mysqli_connect_error() . "
        </div>");
}

// Banner helper used for status display
$db_status_banner = "
    <div style='background-color: #c6f6d5; color: #22543d; border: 1px solid #9ae6b4;
                padding: 10px; border-radius: 4px; text-align: center; font-weight: bold; margin-bott
        ✔ Database String Status: Connected Successfully
    </div>";
?>
```

FIGURE5: GITHUB UPDATES



DESCRIPTION

This week I organized my codes into separate folders to have a folder tree like structure in vs code for easier access as shown in figure 3. I focused on using SHA-256 password hashing to prevent security loopholes.

WEEK5: DATABASE CREATION AND CRUD OPERATIONS
DATA DRIVEN SYSTEMS USING DATABASES

FIGURE1: DATABASE& TABLE CREATION

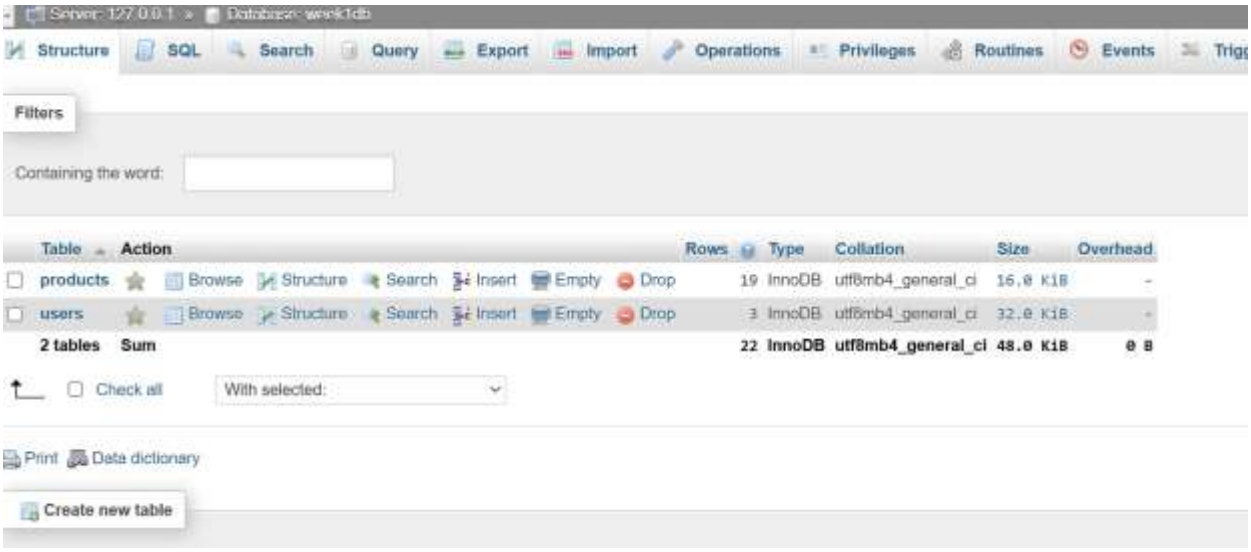


FIGURE2: CRUD OPERATIONS

```php
// 🟩 ADMIN PRIVILEGE: Process Form Submissions (Create / Update)
if ($_SERVER["REQUEST_METHOD"] == "POST" && $is_admin) {
    if (isset($_POST['action']) && $_POST['action'] == 'update') {
        $id = intval($_POST['id']);
        $name = mysqli_real_escape_string($conn, $_POST['product_name']);
        $category = mysqli_real_escape_string($conn, $_POST['category']);
        $price = floatval($_POST['price']);
        $stock = intval($_POST['stock_quantity']);
        mysqli_query($conn, "UPDATE products SET product_name='$name', category='$category', price='$
        header("Location: dashboard.php?msg=Record Updated Successfully"); exit();
    } elseif (isset($_POST['product_name'])) {
        $name = mysqli_real_escape_string($conn, $_POST['product_name']);
        $category = mysqli_real_escape_string($conn, $_POST['category']);
        $price = floatval($_POST['price']);
        $stock = intval($_POST['stock_quantity']);
        mysqli_query($conn, "INSERT INTO products (product_name, category, price, stock_quantity) VAL
        header("Location: dashboard.php?msg=Record Created Successfully"); exit();
    }
}
// 🟥 ADMIN PRIVILEGE: Delete Handler
if (isset($_GET['delete_id']) && $is_admin) {
    $delete_id = intval($_GET['delete_id']);
    mysqli_query($conn, "DELETE FROM products WHERE id = $delete_id");
    header("Location: dashboard.php?msg=Record Deleted Successfully"); exit();
}
```

FIGURE3: SQL QUERY EXECUTION

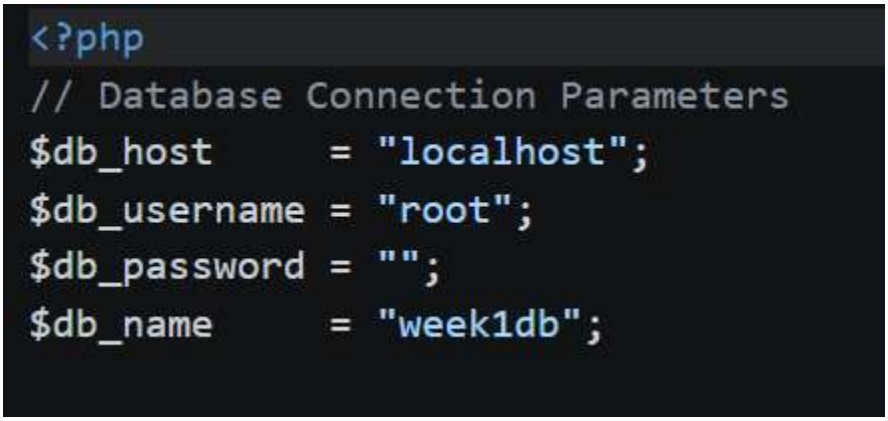
```php
// Query the users table for matching credentials
$query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";
$result = mysqli_query($conn, $query);
```

FIGURE4: LOGIN DATABASE INTEGRATION

```php
// Query the users table for matching credentials
$query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";
$result = mysqli_query($conn, $query);

if ($result && mysqli_num_rows($result) > 0) {
    // Logged in successfully! Save user to session global tracking
    $_SESSION['user'] = $username;
```

FIGURE5: PHP DATABASE CONNECTION



```php
<?php
// Database Connection Parameters
$db_host     = "localhost";
$db_username = "root";
$db_password = "";
$db_name     = "week1db";
```

DESCRIPTION

This week I built a relational database and created product and users tables with structure columns like ID'S and usernames. Technologies used: MYSQL and phpMyAdmin.

Figure 3 shows code fetching real database rows to decide whether to grant or deny system entry.

I also committed my week 5 to github.

I successfully executed CRUD operations as shown in figure 2 by combining front end template forms with server processing statements to view and erase database parameters securely.